

Answer Key — Final Exam SS 2024 (26 August 2024)

This is a reconstructed key, not an official one. ETH published no solutions for this paper. Every number below was recomputed in Python and every prose answer was checked against the lecture slides (cited as L_XX + slide number). Where the slides are genuinely ambiguous, it says so — those are the only places you should distrust this document.

Note this paper’s **different shape**: a 19-point MC block and unevenly weighted questions (Speech Synthesis is worth 18, Affective Computing only 10). The 2025 paper moved to a flat 13 per question. Expect 2026 to look like one of the two; the *content* is nearly identical either way.

Q	Topic	Max
1	Multiple Choice	19
2	Affective Computing	10
3	Attention Mechanism	10
4	Mel Filter Banks	12
5	Chatbots	17
6	Speech Synthesis	18
7	Knowledge Graphs	10
8	Reinforcement Learning	10
9	Animation	14

Q1 — Multiple Choice (19 pts)

+1 correct, −1 wrong, 0 blank, block floors at 0.

	Statement (abbreviated)	Answer	Why
a	The Mel scale is a linear map from frequency to pitch	False	It is logarithmic : $m = 1127 \ln(1 + f/700)$.
b	RNN-Transducer is offline-only, no streaming	False	RNN-T is <i>the</i> streaming architecture — encoder + prediction + joint network, no full-utterance attention.
c	James-Lange: stimulus leads simultaneously to emotion and bodily change	False	That describes Cannon-Bard . James-Lange is sequential: body first, emotion inferred from it.
d	SC decomposed into phasic and tonic by convex optimization	True	L_02 s.49 (cvxEDA).
e	Positional encoding gives the model token-order information	True	Self-attention is otherwise permutation-invariant.
f	RNNs handle token sequences, e.g. language modeling and sequence classification	True	L_03.
g	Pruning removes the most impactful connections	False	L_05 s.73: “Remove least impactful connections, retrain, repeat.”

	Statement (abbreviated)	Answer	Why
h	Quantization reduces both model size and accuracy	True	L_05 s.76–77: reducing weight precision shrinks the model, and “round-to-nearest degrades performance” — which is why AWQ/SmoothQuant exist.
i	PEFT operates only on a true subset of the model	True (<i>uncertain</i>)	L_05 s.70 frames PEFT as “fine-tune only some parameters”, which reads as True. But LoRA trains <i>newly added</i> matrices, not a subset of existing weights — and the 2025 paper’s own wording says “a small number of additional parameters”. Genuinely ambiguous; if it reappears, consider leaving it blank rather than risking −1.
j	Griffin-Lim reconstructs phase iteratively, no learned parameters	True	L_09 s.26: “Pure signal processing / No learned parameters.”
k	A nuclear accent is the last accent in a phrase and more prominent	True	L_08 s.28, almost verbatim.
l	In FastSpeech, doubling the predicted duration speeds up speech	False	L_09 s.14: $2 \times D = \mathbf{0.5 \times speed}$, i.e. slower. More frames per phoneme = longer sound.
m	Autonomous agents: independent, reactive, proactive, control internal states	True	L_12 s.4, verbatim (reactivity, autonomy, proactiveness).
n	Dialogue trees offer high flexibility for unexpected queries	False	The opposite — inflexibility is their defining weakness. The “hard to scale” half is true, which is the trap.
o	RL rewards must be differentiable so we can backprop	False	L_14 s.16, verbatim: “The reward is not necessary to be differentiable.” This is the entire reason RL exists rather than supervised learning.
p	PPO needs collected trajectories from interaction; they can vary in length	True	PPO is on-policy, and episodes end at different times.
q	“Slow In and Slow Out” emphasizes linear motion	False	It means the opposite: ease in and out of the extremes. Linear motion is what the principle exists to avoid.

	Statement (abbreviated)	Answer	Why
r	Facial muscles attach to bone, connect to skin, wrinkles form perpendicular to contraction	True	Standard facial-anatomy statement from L_10.
s	LBS handles large joint rotations without artifacts	False	Large rotations and twists are exactly where LBS breaks — the “ bow tie ” effect (volume loss).

Score: (correct) – (wrong), floor at 0, max 19.

NOTES

Q2 — Affective Computing (10 pts)

a) Action units, fidgeting, mouth aspect ratio (3 pts)

Action Units (1 pt): from the **Facial Action Coding System (FACS)** — the smallest visually distinguishable movements of individual facial muscles, 46 of them, covering the eye, cheek, nose, mouth and chin regions (L_02 s.36). Each is scored for **presence** and **intensity** $[0, 5]$ per frame (OpenFace). Emotions are read from **combinations**: happiness = AU6 + AU12 (cheek raiser + lip corner puller); surprise = AU1 + AU2 + AU5 + AU26.

Fidgeting (1 pt) — captures *all* movement in the video (body and face), via background subtraction (L_02 s.44):

1. $f_{\text{temp}} = f_{\text{gray}} - b_{\text{gray}}$ — subtract the running background from the current frame
2. **Binarize** f_{temp} with a threshold t
3. $E = \text{percentage of surviving pixels}$ — that is the fidgeting/energy score
4. Update the background: $b_{\text{gray}} \leftarrow (1 - \alpha) b_{\text{gray}} + \alpha f_{\text{gray}}$

Then take basic statistics over frames (mean, std, min, max, and the position of the min/max-energy frame).

Mouth Aspect Ratio (1 pt) — from the 68 facial landmarks, the ratio of vertical mouth openings to horizontal mouth width, averaged over three vertical measurements (L_02 s.43):

$$\text{MAR} = \frac{\|p_2 - p_8\| + \|p_3 - p_7\| + \|p_4 - p_6\|}{3 \|p_5 - p_1\|}$$

b) What HRV and SC measure (2 pts)

HRV (1 pt): heart rate variability — the variation in the time between consecutive heartbeats (**R-R intervals**), an index of **autonomic nervous system** activity (sympathetic vs parasympathetic balance). Time-domain measures: standard deviation of R-R intervals, max–min, pNN50, RMSSD; frequency-domain: LF/HF ratio (LF 0.04–0.1 Hz, HF 0.1–0.5 Hz) (L_02 s.48). **Higher** HRV = relaxed and adaptive; **lower** = stress. Confounded by age, posture, activity and breathing.

Skin conductance (1 pt): sweat-gland activity changing the electrical conductivity of the skin — a direct index of **arousal** (not valence), which is why it is used in lie detectors (L_02 s.49). It splits into a fast, event-related **phasic** component and a slow **tonic** baseline, separated by convex optimization.

c) Mark the phasic features + identify the trigger image (5 pts)

The phasic component is the **orange** curve. Mark on it (1 pt each):

#	Feature	What to draw
1	Amplitude	A vertical arrow from the response onset level up to the peak of the phasic curve.
2	Latency	A horizontal arrow from stimulus onset (left edge of the green window) to the onset of the rise .
3	Rise time	A horizontal arrow from the response onset to the peak .
4	Half-recovery time	A horizontal arrow from the peak to the point where the curve has decayed to half the amplitude .

Getting 2 and 3 the wrong way round is the common error: latency ends where rise time begins.

Which trigger image (1 pt): Image B, the kitten. The signal shows a **large, sharp phasic response** during the stimulus window, and skin conductance indexes **arousal**. An emotionally engaging picture (the animal) produces a strong SCR; the neutral picture of a desk and chairs (Image A) is low-arousal and would produce little or no phasic response. Note SC does **not** tell you valence — only that the image was arousing.

Q3 — Attention Mechanism (10 pts)

$\mathbf{q} = [1, 0, 1]^\top$; $\mathbf{x}_1 = [1, 0, 0]^\top$, $\mathbf{x}_2 = [0, 1, 0]^\top$, $\mathbf{x}_3 = [1, 1, 0]^\top$; $d = 3$, so $\sqrt{d} = 1.732$.

a) Similarity scores (3 pts)

$$s_t = \frac{\mathbf{q}^\top \mathbf{x}_t}{\sqrt{d}}$$

Dot products: $\mathbf{q}^\top \mathbf{x}_1 = 1$, $\mathbf{q}^\top \mathbf{x}_2 = 0$, $\mathbf{q}^\top \mathbf{x}_3 = 1$.

$$s = \left[\frac{1}{1.732}, \frac{0}{1.732}, \frac{1}{1.732} \right] = [\mathbf{0.577}, \mathbf{0}, \mathbf{0.577}]$$

$\mathbf{x}_1$ and $\mathbf{x}_3$ each overlap the query in exactly one dimension; $\mathbf{x}_2$ is orthogonal to it.

Ambiguity worth knowing: the paper says “scaled dot-product attention”, so divide by $\sqrt{d}$. If you omit the scaling you get $s = [1, 0, 1]$ and eventually $\mathbf{c} = [0.845, 0.578, 0]$ — a defensible reading, but write down which convention you used.

b) Attention weights (3 pts)

$$a_t = \frac{e^{s_t}}{\sum_i e^{s_i}}$$

$$e^{0.577} = 1.781, \quad e^0 = 1.000, \quad e^{0.577} = 1.781, \quad \sum = 4.562$$

$$a = \left[\frac{1.781}{4.562}, \frac{1.000}{4.562}, \frac{1.781}{4.562} \right] = [\mathbf{0.390}, \mathbf{0.219}, \mathbf{0.390}]$$

$$\sum_t a_t = 0.390 + 0.219 + 0.390 = 0.999 \approx 1 \checkmark$$

Always state this check. If your weights don't sum to 1, say so on the paper and continue — the method marks survive an arithmetic slip; an unflagged impossible answer looks worse.

c) Context vector (4 pts)

$$\mathbf{c} = \sum_t a_t \mathbf{x}_t$$

$$\mathbf{c} = 0.390 \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + 0.219 \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + 0.390 \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0.390 + 0 + 0.390 \\ 0 + 0.219 + 0.390 \\ 0 \end{bmatrix} = \begin{bmatrix} \mathbf{0.781} \\ \mathbf{0.609} \\ \mathbf{0} \end{bmatrix}$$

The third component is 0 because **no input vector has a third component** — attention can only ever return a point inside the convex hull of the values it is averaging.

NOTES

Q4 — Mel Filter Banks (12 pts)

200–8000 Hz, 3 filters, 5 points. Round to the nearest integer.

$$m = 1127 \ln\left(1 + \frac{f}{700}\right), \quad f = 700(e^{m/1127} - 1)$$

Step 1 — Convert both bounds to Mel (2 pts)

$$m_{\text{low}} = 1127 \ln\left(1 + \frac{200}{700}\right) = 1127 \ln(1.2857) = 1127 \times 0.2513 = \mathbf{283.23}$$

$$m_{\text{high}} = 1127 \ln\left(1 + \frac{8000}{700}\right) = 1127 \ln(12.4286) = 1127 \times 2.5200 = \mathbf{2840.04}$$

Step 2 — Five points spaced linearly in Mel (3 pts)

Using the formula the paper hands you, $\text{Point}_i = a + i \cdot \frac{b-a}{4}$:

$$\frac{2840.04 - 283.23}{4} = 639.20$$

i	0	1	2	3	4
Mel	283.23	922.43	1561.63	2200.84	2840.04

Step 3 — Convert every point back to Hz (4 pts)

$$f_i = 700(e^{m_i/1127} - 1)$$

i	0	1	2	3	4
Mel	283.23	922.43	1561.63	2200.84	2840.04
Hz	200	887	2098	4234	8000

Free self-check: f_0 and f_4 must come back out as your original bounds, 200 and 8000. If they don't, you inverted the formula wrong — fix it before continuing.

Step 4 — Assemble the three filters (3 pts)

Each filter takes three **consecutive** points as (start, peak, end), **overlapping by one**:

Filter	Start	Peak	End
1	200	887	2098
2	887	2098	4234
3	2098	4234	8000

Each is **triangular**: weight 0 at the start, 1 at the peak, 0 at the end.

Say this for the last mark: the spacing is uniform in **Mel** and therefore **non-uniform in Hz** — filter 1 spans 1898 Hz, filter 3 spans 5902 Hz. Narrow at low frequencies, wide at high ones. That mirrors human hearing, where pitch resolution is far finer at low frequencies, and is the entire point of using the Mel scale.

NOTES

Q5 — Chatbots (17 pts)

a i) Which sampling strategy could produce “price”? (3 pts)

“price” is ranked **third** at $P = 0.20$. Greedy decoding is therefore ruled out — it would always emit “company” (0.35). Name **one** of these and explain it, **with parameters** (L_05 s.38–42):

- **Top-k sampling** with $k \geq 3$: keep the k highest-probability tokens, renormalize, sample randomly among them. With $k = 3$ the candidates are {company, bank, price} and “price” is reachable.
- **Top-p (nucleus) sampling** with $p \geq 0.80$: keep the smallest set whose cumulative probability reaches p . Here $0.35 + 0.25 + 0.20 = 0.80$, so any $p > 0.60$ already includes “price”.
- **Temperature sampling** with a **high temperature** $T > 1$: flattens the distribution before sampling, raising the chance of lower-ranked tokens.

Marking: 1 pt naming a valid strategy, 1 pt explaining the mechanism, 1 pt specifying the parameter value and showing it admits “price”. **The parameter is a third of the marks** — the question asks for it explicitly.

a ii) Output is repetitive — how to increase diversity? (2 pts)

Action (1 pt) + justification (1 pt). Any of:

- **Increase k (or p):** a larger candidate set means more tokens can be sampled, so the model stops falling into the same high-probability groove every time.
- **Raise the temperature:** flattens the distribution, shifting mass toward lower-ranked tokens and making generation less deterministic.
- **Switch top-k → top-p:** the nucleus adapts to the distribution’s shape, so at flat, genuinely uncertain steps it admits more options rather than a fixed k .
- **Add a repetition/frequency penalty:** down-weight tokens already generated.

The trade-off, worth stating: too much diversity costs coherence — L_05 s.39 warns that “too high a k can disrupt coherence.” Mentioning the trade-off is what distinguishes a 2/2 answer.

b i) Three disadvantages of fine-tuning vs prompt engineering (3 pts)

One point each:

1. **Cost:** needs labelled task data plus significant compute and training time. Prompting is free and works immediately.
2. **Catastrophic forgetting / overfitting:** updating the weights can degrade the model’s general ability, especially on a small dataset.
3. **Storage and maintenance:** you keep a **separate model copy per task**, and everything must be redone whenever the base model is updated. A prompt is a string you edit in a second.

Also acceptable: needs ML expertise and infrastructure; not possible for API-only models; harder to iterate on and debug; the resulting model is static and cannot incorporate new knowledge without retraining.

b ii) Three prompt engineering strategies (6 pts)

Two points each — **name it and explain how it works**. From L_06:

1. **System prompts / role assignment** (s.4): chat models take a **system** message that runs once before the conversation and sets the **persona, constraints and output format** (“You are Einstein, a friendly physicist. Answer only physics questions. Be concise.”). It shapes every subsequent turn without touching the weights.
2. **In-context / few-shot learning** (s.5, s.10): put **worked examples** of the task directly in the prompt (input→output pairs). The model infers the pattern and applies it to the new input, improving performance **without any gradient updates**. Choosing which examples to include is itself an optimization problem (DSPy, random search, Bayesian optimization).
3. **Chain-of-Thought** (s.9): instruct the model to produce **intermediate reasoning steps** before its answer (“think step by step”), which improves multi-step and arithmetic tasks. Its weakness is that the reasoning is ungrounded, so the model can hallucinate confidently.
4. **ReAct** (s.9): interleaves **Thought** → **Action** → **Observation** loops, letting the model call external tools/APIs and update its plan on the results. Fixes CoT’s grounding problem.
5. **Structured outputs / function calling** (s.8): constrain the response to a fixed schema (JSON), or expose tool definitions the model can invoke with structured arguments.

Any three, correctly explained.

b iii) What’s wrong with the colleague’s output? (3 pts)

What’s wrong (1 pt): the model **does not stop**. Instead of answering the single question and halting, it emits the answer and then **continues generating further Question/Answer/Hint blocks of its own invention** — inventing new questions (“count the number of spaces”, “count the number of tokens”) and answering them. The desired output was one line, **The answer is 5**; the actual output is a runaway continuation of the document pattern. (It also fails to match the requested phrasing.)

Why (1 pt) — this is what the hint points at: the model was trained on **raw text with a next-token-prediction objective**, not on instruction–response pairs. A base language model’s only skill is *continuing a document*, and a document containing one Question/Answer/Hint block most plausibly continues with another. It has no notion of “the task is over now.”

Recommended action (1 pt): instruction tuning — fine-tune (or switch to a model already fine-tuned) on **instruction–response pairs**, so the model learns to respond to an instruction and terminate (L_06 s.5). Practical alternatives that earn the mark: define a **stop sequence** so generation halts at the next **Question:**, or supply few-shot examples that demonstrate the exact desired format *and* its termination.

NOTES

Q6 — Speech Synthesis (18 pts)

a i) What is a diphone and why preferred over phones? (2 pts)

Definition (1 pt): a diphone runs from the **middle of one phone to the middle of the next** (L_08 s.36).

Why (1 pt): coarticulation. Every phone is acoustically altered by its neighbours — the second formant of [eh] depends on the phones either side. The **edges** of a phone are where that contextual distortion lives; the **middle is the most stable** part. Cutting mid-phone therefore (a) captures the **transition between sounds inside the unit**, which is the hard part to synthesize, and (b) puts the **join in a stable region** where two units can be concatenated without an audible discontinuity. Concatenating whole phones would put every seam exactly where the signal is changing fastest.

Useful number: 39 phones $\rightarrow 39^2 = 1521$ possible diphones, of which about **1172** actually occur, roughly 8 MB of audio for English.

a ii) Target and join costs (3 pts)

Target cost (1 pt) — “*is this unit the right sound?*” For a target diphone s_t and a candidate database unit u_j :

$$T(s_t, u_j) = \sum_{n=1}^N w_n T_n(s_t, u_j)$$

The sum runs over **features**, not units: N features, each with its own mismatch T_n and weight w_n . The slides list F_0 , stress, duration, position.

Join cost (1 pt) — “*will it splice cleanly onto the next one?*”

$$J(u_j, u_{j+1}) = \sum_{p=1}^P w_p J_p(u_j, u_{j+1})$$

over P join features: F_0 , energy, spectral features. **Crucially** $J(u_j, u_{j+1}) = 0$ **when the two units were consecutive diphones in the original recording** — no seam, nothing to smooth. Say this; it is stated explicitly on L_08 s.43.

How the best sequence is found (1 pt): not greedily, unit by unit — you minimize **both costs jointly over the whole utterance** (L_08 s.44):

$$\hat{U} = \arg \min_U \sum_{j=1}^J T(s_j, u_j) + \sum_{j=1}^{J-1} J(u_j, u_{j+1})$$

J units, so $J - 1$ seams. (The lecture reuses the letter J for both the join-cost function and the number of units — that’s their notation, not a typo.) In practice this is a Viterbi search over the lattice of candidate units.

a iii) Diphone vs unit selection: one advantage, one disadvantage (2 pts)

Advantage of diphone synthesis (1 pt): a **tiny, bounded database** — about 1172 diphones, a few minutes of recorded speech, ~8 MB, recorded from one speaker in a single session. Unit selection needs around **10 hours** of speech. Diphone synthesis is also fully predictable: coverage is guaranteed because every diphone is present exactly once.

Disadvantage (1 pt): quality. With one copy of each diphone, prosody must be forced onto it by **signal processing** (pitch/duration modification), which **leaves audible artifacts**, and the unit only captures **local** coarticulation between immediate neighbours (L_08 s.42). Unit selection keeps many copies of each unit in different contexts, so it can usually *find* a unit with roughly the right prosody and apply little or no signal processing — much more natural output.

b i) Tacotron 2 components (4 pts)

One point each.

Encoder — convolutional layers (1 pt): 3 layers of **512 filters, length 5**, over the character embeddings. The width-5 kernel gives each position a receptive field covering the **two preceding and two following characters**, capturing local letter context (L_09 s.7).

Encoder — bi-directional LSTM (1 pt): processes the convolution outputs **forwards and backwards** and concatenates the two hidden states, so every encoder output encodes information from **both past and future** characters in the sentence — necessary because pronunciation and prosody depend on what comes after a character as much as before.

Decoder — postnet (1 pt): convolutional layers followed by a feed-forward net (80 neurons) applied **after** the decoder’s mel prediction y_i , producing a refined $y_{\text{final},i}$. It predicts a **residual correction** that improves output quality; the loss sums the error **before and after** the postnet:

$$L = \frac{1}{n} \sum_i (y_{\text{real},i} - y_i)^2 + \frac{1}{n} \sum_i (y_{\text{real},i} - y_{\text{final},i})^2$$

Decoder — reduction factor r (1 pt): the decoder predicts r **mel frames per step** instead of one, cutting the number of autoregressive steps by a factor of r . This **speeds up computation and reduces memory usage** — and in practice helps attention converge, since there are fewer steps to align.

b ii) FastSpeech duration modeling + two advantages (4 pts)

Explicit duration modeling (2 pts) (L_09 s.12–14):

- One phoneme corresponds to **several mel-spectrogram frames**, and how many is what an autoregressive attention has to figure out implicitly.
- A **duration predictor** outputs, for each phoneme, how many frames it should occupy — e.g. for *knight*, $D = [1, 8, 15, 3, 0, 17]$ (note the 0: a silent letter gets zero frames).
- A **length regulator** then **upsamples the encoder states by repeating each one D_i times**, so the encoder output is stretched to exactly the target mel length **before** the decoder runs. The decoder is a feed-forward Transformer and generates every frame **in parallel**.
- The duration predictor is trained on durations extracted from a **teacher TTS model’s** attention alignments.

Two advantages over Tacotron 2 (1 pt each):

1. **Much faster inference** — non-autoregressive, all frames generated in parallel, instead of a sequential loop over hundreds of frames.
2. **Robustness** — no attention to fail, so it eliminates Tacotron 2’s **word skipping and repetition** (L_09 s.11).
3. **Controllability** — scaling the durations directly controls **speaking rate** ($2D = 0.5 \times \text{speed}$, $0.5D = 2 \times \text{speed}$), and inserting durations at punctuation controls **breaks**. Tacotron 2 offers no such handle.

Any two.

b iii) WaveNet vs HiFiGAN, and which is better for real time (3 pts)

WaveNet (1 pt): **autoregressive** — generates **one audio sample at a time**, each conditioned on all previous samples. Built from **causal dilated convolutions** (dilation lets the receptive field grow exponentially with depth, capturing long-range temporal structure) with a **softmax output** — the model *classifies* over 8-bit mu-law levels rather than regressing a continuous value (L_09 s.27).

HiFiGAN (1 pt): a GAN, trained as generator + discriminator. The generator is **fully parallel: transposed-convolution upsampling layers** raise the mel spectrogram to waveform resolution, with **residual blocks of dilated convolutions** and **Multi-Receptive Field Fusion** (parallel residual blocks at different dilation rates, outputs fused). It is judged by a **Multi-Scale Discriminator** and a **Multi-Period Discriminator** (the latter targets the periodic structure of speech, so pitch and tone come out accurate), trained with adversarial + feature-matching + mel-spectrogram L1 losses (L_09 s.31–35).

Which for real time, and why (1 pt): **HiFiGAN**. WaveNet’s autoregressive loop needs one forward pass **per sample** — at 22 kHz that is 22 000 sequential network evaluations per second of audio, far too slow for real-time use however good it sounds. HiFiGAN produces the entire waveform in a **single parallel forward pass**, giving “excellent sound quality at low computational cost” (L_09 s.31) — which is why Microsoft Azure TTS uses a vocoder of this type. L_09 s.46 states the conclusion outright: **HiFiGAN is the best trade-off between quality and speed**.

Q7 — Knowledge Graphs (10 pts)

The graph in Fig. 4: ETH Zurich `-is_a→` Public Research University, `-located_in→` Zurich, `-focus_on→` STEM, Zurich `-is_in→` Switzerland, and ETH Zurich `EPFL` labelled `sister_institution` in both directions.

a) Standard format + one advantage of KG + LLM (2 pts)

Format (1 pt): RDF triples — (subject, predicate, object), equivalently (head, relation, tail). Nodes are entities, edges are relationships; the graph is labelled, directed, multi-relational and heterogeneous (L_12 s.28–31).

Advantage of integrating with an LLM (1 pt): it grounds the model in verified facts, reducing hallucination (L_12 s.8 says this explicitly). The knowledge graph supplies **factual accuracy** and **up-to-date, structured knowledge** that can be updated without retraining the model, and it makes answers **traceable** to a source triple.

b) One relation type + structured querying + the massive/incomplete case (3 pts)

One type of relation (1 pt). Answer in the lecture’s taxonomy (L_12 s.43), and this graph gives you a gift:

- **Symmetric: sister_institution** — the figure draws arrows in **both directions** between ETH Zurich and EPFL. This is the clearest answer and the one the figure was built to elicit.
- Also correct: **asymmetric** (`is_a`, `focus_on`, `located_in` — ETH is a public research university, but not vice versa); **composition** (`located_in` then `is_in` composes to “ETH is in Switzerland”).

Structured querying method (1 pt): SPARQL — the W3C standard for querying RDF graphs. A query is a **graph pattern** of triple patterns in which any of subject, predicate or object may be a variable (L_12 s.33).

If the graph is massive and incomplete (1 pt): knowledge graph embeddings (KGE), e.g. **TransE** — embed entities and relations as vectors in $\mathbb{R}^d$ so that $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ for true triples. This enables **link prediction** of edges the graph never recorded, which SPARQL fundamentally cannot do: SPARQL operates only on explicitly stored, static data and **cannot infer unknown relationships** (L_12 s.38–39).

c) Query type + how each approach answers Q (5 pts)

Q = “Are there any public research universities in Zurich focusing on STEM?”

Query type (1 pt): a conjunctive query. Three conditions AND-ed on one variable — `is_a` Public Research University and `located_in` Zurich and `focus_on` STEM. (A single condition would be one-hop; a chain like `ETH → Zurich → Switzerland` would be a path query.)

Structured approach — the graph pattern to look for (2 pts): find every node `?x` such that **all three** of these edges exist simultaneously, sharing the same `?x`:

```
?x is_a      "Public Research University"
?x located_in "Zurich"
?x focus_on  "STEM"
```

Take the **intersection** of the three result sets. Each condition is one one-hop pattern; linking them by **reusing the same variable** is what expresses the AND (L_12 s.36). Against Fig. 4 this returns **ETH Zurich**. (No formal syntax required, but the shared-variable structure must be visible.)

Embedding approach — the steps (2 pts):

1. **Train** a KGE model (TransE) on the graph, learning a vector for every entity and every relation such that $\mathbf{h} + \mathbf{r} \approx \mathbf{t}$ holds for the observed triples.
2. **Embed the query** as vector arithmetic: for each condition, compute the target point $\mathbf{t} = \mathbf{h} + \mathbf{r}$ — e.g. $\mathbf{q}_1 = \text{PublicResearchUniversity} - \mathbf{r}_{\text{is_a}}$, and similarly for `located_in` Zurich and `focus_on` STEM. (Or, for a conjunctive query, use **Query2Box**: project each anchor into a box per relation hop and **intersect** the boxes.)
3. **Score every candidate entity** by distance, $\|\mathbf{h} + \mathbf{r} - \mathbf{t}\|$, summed or intersected across the three conditions.
4. **Rank and threshold** — return the nearest entities as answers.

The point of the contrast, worth stating: the embedding approach returns **plausible answers even when the edges are missing from the graph**. If the `focus_on` STEM triple had never been recorded, SPARQL would return nothing while the embedding still ranks ETH Zurich highly. The cost is that results are **approximate and unranked-by-certainty** rather than exact.

NOTES

Q8 — Reinforcement Learning (10 pts)

Setting: ball trajectory $(x_0, x_1, \dots, x_t)$ fixed and known; player dynamics $y_t = f(y_{t-1}, z_t)$; player starts within 1 m of x_0 ; goal is to **reach the moving ball as soon as possible**. Ball and player are synchronized in time.

a) Two elements of the state, and the action (3 pts)

State s_t (2 pts — one per element): the state must contain everything relevant to the goal (L_14 s.50), and here the goal is *moving*:

1. **The player's own location y_t** (its current state / motion seed).
2. **The football's location x_t** — or, better, the **relative displacement** $x_t - y_t$, the vector from player to ball.

Only y_t is not enough: the ball moves, so without ball information the policy cannot know which way to run. Including the next waypoint x_{t+1} or the ball's velocity is a legitimate refinement.

Action a_t (1 pt): the **control signal** z_t — the continuous latent variable of the player's cognitive/motion model, since that is the input the dynamics $f(y_{t-1}, z_t)$ consumes. Continuous action space.

b) MLP policy: input, output, how to get the action (3 pts)

- **Input (1 pt):** the **state s_t** — the player's location and the ball's location (or their relative displacement).
- **Output (1 pt):** because the policy is **stochastic**, the network does **not** output an action directly. It outputs the **parameters of a probability distribution over actions** — for a continuous action space, the **mean $\mu(s_t)$ and standard deviation $\sigma(s_t)$ of a Gaussian** (L_14 s.14: continuous action space $\rightarrow$ the policy is a **regressor**, not a classifier).
- **How to get the action (1 pt):** **sample** from that distribution, $a_t \sim \pi(\cdot | s_t) = \mathcal{N}(\mu(s_t), \sigma(s_t)^2)$. At evaluation time you may instead take the mean for deterministic behaviour; during training you must sample, because the stochasticity is the exploration.

c) Two elements of the reward (2 pts)

The goal is *reach the ball and as soon as possible*, so the reward needs one term for each (1 pt each):

1. **A goal/proximity term** — reward for reducing the distance to the ball, e.g. $-\|y_t - x_t\|$, or the improvement $\|y_{t-1} - x_{t-1}\| - \|y_t - x_t\|$, or a large bonus when $\|y_t - x_t\| < \epsilon$ (contact).
2. **A time penalty** — a constant negative reward $-c$ per step, so every extra time step costs something and the policy is pushed to reach the ball early. (The discount factor $\gamma < 1$ in the return does part of this job too, but the paper asks for two reward *elements*.)

A **motion-realism / control-effort** term is also a defensible second element, given the lecture's own GAMMA formulation — but “as soon as possible” in the prompt is pointing at the time penalty.

d) The $\mathcal{N}(0, I)$ prior as a loss term (2 pts)

Since z_t is the latent variable of a **pre-trained motion model** whose latent space is standard normal, the policy must not wander outside that space or it will produce unrealistic motion. Encode this as a **KL-divergence regularizer between the policy's action distribution and the prior** (L_14 s.59, verbatim from the lecture's own loss):

$$\mathcal{L}_{\text{policy}} = \underbrace{\mathcal{L}_{\text{PPO}}}_{\text{policy update}} + \underbrace{\mathbb{E}[(R_t - V(s_t))^2]}_{\text{value function}} + \underbrace{\alpha \Psi(\text{KL}(\pi(z | s) \parallel \mathcal{N}(0, I)))}_{\text{motion prior}}$$

1 pt for identifying it as a KL divergence to $\mathcal{N}(0, I)$ (a **motion prior encouraging realism**), **1 pt** for the expression. α weights the term against the task reward; Ψ is a clamping/annealing function so the prior does not dominate. A plain $\text{KL}(\pi(z|s) \parallel \mathcal{N}(0, I))$ term, or the equivalent $\mathbb{E}[\|z_t\|^2]$ penalty, should also earn the marks.

Q9 — Animation (14 pts)

Two links $L_1 = 5$, $L_2 = 4$; $\theta_1^0 = \theta_2^0 = \pi/2$; target $\mathbf{e}^* = [5, 5]^\top$; $\alpha = 0.1$. **Radians.**

Cumulative angles: $\theta_1 = \pi/2 = 90^\circ$, $\theta_1 + \theta_2 = \pi = 180^\circ$.

$$\cos : 0, -1 \qquad \sin : 1, 0$$

a i) Initial end-effector position (2 pts)

$$\mathbf{e}^0 = \begin{bmatrix} L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix} = \begin{bmatrix} 5(0) + 4(-1) \\ 5(1) + 4(0) \end{bmatrix} = \begin{bmatrix} -4 \\ 5 \end{bmatrix}$$

The upper arm points straight up (5 units in y), the forearm folds back horizontally (4 units in $-x$). Sketch it — the geometry confirms the algebra in five seconds.

a ii) Error (1 pt)

$$\Delta \mathbf{e} = \mathbf{e}^* - \mathbf{e}^0 = \begin{bmatrix} 5 - (-4) \\ 5 - 5 \end{bmatrix} = \begin{bmatrix} 9 \\ 0 \end{bmatrix}$$

The hand is already at the right height; it needs to travel 9 units in $+x$.

a iii) Jacobian (4 pts)

$$J = \frac{\partial \mathbf{e}}{\partial \theta} = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) & -L_2 \sin(\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) & L_2 \cos(\theta_1 + \theta_2) \end{bmatrix}$$

Column 2 drops the L_1 terms — rotating the elbow cannot move the upper arm.

$$J = \begin{bmatrix} -5(1) - 4(0) & -4(0) \\ 5(0) + 4(-1) & 4(-1) \end{bmatrix} = \begin{bmatrix} -5 & 0 \\ -4 & -4 \end{bmatrix}$$

Free check: row 2 column 1 must equal $e_x^0 = -4$, and it does. Also, the J^+ printed on the paper is the exact inverse of this J — if your J doesn't invert to $\begin{bmatrix} -0.2 & 0 \\ 0.2 & -0.25 \end{bmatrix}$, you made an error. **Use that: the paper is handing you the answer key to part (iii).**

a iv) Change in rotation angles (2 pts)

$$\Delta \theta = \alpha J^+ \Delta \mathbf{e}$$

$$J^+ \Delta \mathbf{e} = \begin{bmatrix} -0.2 & 0 \\ 0.2 & -0.25 \end{bmatrix} \begin{bmatrix} 9 \\ 0 \end{bmatrix} = \begin{bmatrix} (-0.2)(9) + (0)(0) \\ (0.2)(9) + (-0.25)(0) \end{bmatrix} = \begin{bmatrix} -1.8 \\ 1.8 \end{bmatrix}$$

$$\Delta \theta = 0.1 \begin{bmatrix} -1.8 \\ 1.8 \end{bmatrix} = \begin{bmatrix} -0.18 \\ 0.18 \end{bmatrix}$$

Do not forget α .

a v) Update the angles (2 pts)

$$\theta^1 = \theta^0 + \Delta \theta = \begin{bmatrix} 1.571 \\ 1.571 \end{bmatrix} + \begin{bmatrix} -0.18 \\ 0.18 \end{bmatrix} = \begin{bmatrix} 1.39 \\ 1.75 \end{bmatrix} \text{ rad}$$

($\pi/2 = 1.5708$.) Shoulder rotates clockwise, elbow counter-clockwise — the arm starts unfolding toward the apple on the right. Stay in radians. (*The paper prints " θ_1^1 " twice in the answer box; the second is θ_2^1 .*)

b i) Manifold learning + one application (2 pts)

The idea (1 pt): human motion is **highly constrained** — joints have limits, limbs have fixed lengths, gaits are cyclic — so although a pose lives in a high-dimensional space (hundreds of DoF), real motion occupies only a thin, **low-dimensional manifold** inside it. Manifold learning finds a mapping

$$f : \mathbb{R}^d \rightarrow \mathbb{R}^m, \quad m < d$$

to the small set of **intrinsic variables** (phase, speed, style, joint configuration) that actually explain the data. Working in that latent space makes synthesis tractable and sidesteps the curse of dimensionality — and it guarantees that any latent point decodes to a **plausible** pose, which sampling raw joint angles would not.

Application (1 pt): motion synthesis and interpolation — smoothly blending between motions along the manifold instead of averaging poses in joint space (which produces impossible in-between poses). Also: motion denoising/completion from sparse sensors, and character control from a low-dimensional signal.

b ii) Method for gesture synthesis (1 pt)

Normalizing flows — L_11 s.32–33, *Style-Controllable Speech-Driven Gesture Synthesis Using Normalizing Flows* (GestureFlow / MoGlow, [ALE20]), built from **affine coupling layers** taken from GLOW. The invertible-flow formulation gives diverse, **style-controllable** co-speech gestures with exact likelihood.

(In the SS2026 deck, **diffusion models** are also presented for this — DiffGesture, audio-conditioned co-speech gesture diffusion, L_11 s.47. Either answer should be accepted; normalizing flows is the one the 2024 slides pointed at.)

NOTES

Self-grading

Add up the sub-question points, then record the total in [progress.md](#). Diagnose by category rather than by total:

- **Lost points on a calculation?** Cheapest fix available. Re-drill: `python3 drill.py <recipe>`.
- **Lost points on prose?** Check [DEFINITIONS.md](#) first — if the answer is there, it's a memorization gap, not a knowledge gap.
- **Lost points on the MC block?** Deck 00_Exam_MC_Traps. Blank beats a guess: -1 vs 0 .

Calculation points available on this paper: Q3 (10), Q4 (12), Q9a (11) = **33**, plus the 19-point MC block = **52**. Slightly short of the 60 you need — this paper's prose weighting is heavier than 2025's, so Q6 (18 pts of speech synthesis, all prose) is where the remaining margin has to come from.

NOTES
